

## Aula 13 - Automação de Testes e Testes Unitários com PHPUnit

**Descrição:** Nesta aula, vamos aprender a substituir os testes manuais e informais por rotinas automatizadas e profissionais utilizando a Pirâmide de Testes. Através de exemplos práticos, abordaremos os conceitos de testes unitários, o padrão *Triple A* (Arrange, Act, Assert) e a documentação de Casos de Testes. Ao final, você aplicará esses conceitos diretamente no projeto da sua equipe (SA), configurando o ambiente via Composer e gerando scripts de teste reais.

---

### CONCEITOS

#### 1. O Problema dos Testes Informais vs. A Necessidade de Automação

Tradicionalmente, os programadores tendem a testar seus códigos por meio de métodos manuais e informais, como a inserção de condicionais (if) temporários ou comandos de impressão direta na tela, a exemplo de `var_dump()` no PHP ou `console.log()` no JavaScript. Essa abordagem traz riscos operacionais graves: há o perigo real de esquecer esses códigos de depuração na versão enviada para produção e a extrema dificuldade de replicar tais testes à medida que o sistema cresce em volume e complexidade.

A automação surge como a solução profissional para esse gargalo, consistindo na execução de rotinas por meio de ferramentas dedicadas que geram scripts reutilizáveis. Em vez de navegar manualmente por formulários e interfaces a cada modificação, o desenvolvedor executa scripts automatizados em segundos para assegurar que novas alterações não quebraram funcionalidades preexistentes. Isso libera o profissional para tarefas mais complexas e confere maior previsibilidade ao projeto.

#### 2. Planejamento Estratégico e a Pirâmide de Testes

A decisão sobre o que e quando automatizar exige rigor técnico. Não se deve automatizar sistemas instáveis ou sob constante mutação de regras e dados, pois o custo de manutenção dos scripts superaria o retorno sobre o investimento (ROI). Para equilibrar a velocidade de execução e o custo de manutenção, a Engenharia de Software preconiza o uso da **Pirâmide de Testes**. No ecossistema PHP, essa pirâmide divide-se em camadas bem definidas:

- **Análise Estática:** Validação de padrões e erros de tipagem sem executar o código, utilizando ferramentas como PHPStan e Psalm.
- **Testes Unitários (Base):** Focam na menor fração isolada do código, garantindo rapidez e baixo custo de manutenção. O framework padrão da indústria para essa camada é o PHPUnit.
- **Testes de Integração (Meio):** Verificam se múltiplas unidades e componentes funcionam adequadamente em conjunto, validando fluxos que conectam controladores, serviços, repositórios e o banco de dados. Ferramentas como o Codeception apoiam essa etapa.
- **Testes E2E e de Comportamento (Topo):** Simulam o fluxo completo sob a perspectiva do usuário final, utilizando frameworks de BDD (Behavior-Driven Development) como o Behat ou ferramentas de automação web como Cypress e Playwright.

Geralmente, os testes focados no código interno (como unidade e integração) pertencem à categoria de **Testes Caixa Branca**, cujo propósito é validar e obter respostas diretas de funções, classes e estruturas de banco de dados. Para que esses testes operem com confiabilidade, o ambiente deve garantir a **Idempotência**, ou seja, assegurar que o cenário de testes sempre retorne ao seu estado original após cada execução, evitando que dados residuais contaminem as próximas rodadas.

#### 3. Fundamentos do Teste Unitário

O teste unitário consiste na verificação da menor fração de código possível de forma totalmente isolada. Na Programação Orientada a Objetos (POO), essa fração isolada reflete-se, essencialmente, nos métodos de uma classe. O principal pré-requisito do teste unitário é o completo **isolamento**: o método sob teste não pode interagir com o banco de dados real, redes ou APIs externas.

Entre as principais vantagens dessa metodologia destacam-se o isolamento rápido de erros, a alta repetibilidade de rotinas, a facilidade de manutenção de rotinas lógicas, a segurança ao realizar refatorações estruturais (*refactoring*) sem o temor de inserir novos bugs e a constituição de uma "documentação viva", dado que os arquivos de testes expressam de forma técnica e atualizada exatamente o comportamento esperado do sistema.

#### 4. O Framework PHPUnit: Pilares e Anatomia

Criado por Sebastian Bergmann, o PHPUnit é a ferramenta padrão e indispensável no ecossistema PHP, servindo de alicerce para metodologias como o TDD (Test-Driven Development) e vindo pré-configurado em frameworks modernos como Laravel e Symfony. O funcionamento prático do framework apoia-se em três pilares fundamentais:

- A. **Casos de Teste (Test Cases)**: Classes de teste que estendem obrigatoriamente a classe base `PHPUnit\Framework\TestCase`, contendo métodos de validação cujos nomes começam com o prefixo `test` ou utilizam a anotação `@test`.
- B. **Asserções (Assertions)**: Métodos que confrontam o resultado gerado pelo código com o resultado esperado (o "oráculo" do teste). Se a asserção for verdadeira, o teste passa (feedback visual do *ponto verde* no terminal); se falsa, o teste falha (exibindo um *F vermelho* acompanhado do motivo da falha). Exemplos comuns incluem `assertEquals()`, `assertTrue()` e `assertInstanceOf()`.
- C. **Ciclo de Vida (Fixtures)**: Métodos voltados à preparação e limpeza do ambiente. O método `setUp()` roda de forma automática antes de cada teste da classe, enquanto o `tearDown()` é acionado após a conclusão do teste para limpar resíduos.

A escrita interna de um teste profissional segue o padrão **Triple A (Arrange, Act, Assert)**:

- **Arrange (Preparação)**: Configuração prévia e instância de objetos necessários.
- **Act (Ação)**: Invocação direta do método ou função que se deseja testar.
- **Assert (Verificação)**: Execução das asserções para validar os resultados obtidos.

A utilização de um ecossistema maduro como o PHPUnit sobrepõe-se à criação de frameworks caseiros baseados em `if/echo` porque automatiza relatórios visuais densos, gera métricas complexas de cobertura de código (*code coverage*), integra-se com pipelines de CI/CD via arquivos XML e fornece ferramentas nativas de **Mocking** para simular de forma segura bancos de dados ou APIs sem alterar registros reais.

#### 5. Configuração, Prática Industrial e Documentação

Para a operacionalização dos testes no ambiente local, é exigido o levantamento de parâmetros de infraestrutura (requisitos de SO, rede, memória, variáveis de ambiente e massa de dados estável), estruturando o ecossistema com o gerenciador de dependências Composer, isolamento por meio de Docker ou Vagrant e parametrização global via arquivo `phpunit.xml`.

Profissionalmente, a automação não caminha dissociada da documentação de Engenharia de Software. As equipes devem mapear os testes lógicos através de **Casos de Teste** estruturados, contendo ID, Título, Pré-condições, Passos de execução, Dados de entrada, Resultado esperado, Resultado obtido e o Status (Aprovado/Reprovado). Atrelar cada caso de teste analítico ao seu respectivo script

automatizado (ex: associar o cenário de login com sucesso ao método `testLoginSucesso()`) é a prática de mercado que garante a rastreabilidade de bugs e a integridade de entregas complexas.

---

## **ATIVIDADE GUIADA e DESAFIO PRÁTICO** - Em anexo

---

### **Conclusão**

Esta aula marca a transição fundamental de práticas amadoras de depuração manual para a engenharia de software profissional através da automação de testes. A essência deste conteúdo demonstra que depender de testes informais traz sérios riscos à medida que o sistema cresce, tornando a automação essencial para garantir a segurança, a reutilização de rotinas e a previsibilidade do projeto.

Em resumo, os principais pilares consolidados no documento são:

- **Estratégia e a Pirâmide de Testes:** A decisão de como testar deve ser guiada pela Pirâmide de Testes, que divide as validações em camadas estruturadas (Análise Estática, Testes Unitários, Integração e E2E) para equilibrar velocidade de execução e custo de manutenção.
- **Foco no Teste Unitário:** A base dessa pirâmide exige o isolamento completo da menor fração de código possível (como métodos de uma classe), garantindo que a lógica seja validada sem interferência de bancos de dados ou APIs externas.
- **O Framework PHPUnit:** Apresentado como a ferramenta indispensável do ecossistema PHP, ele operacionaliza os testes de forma automatizada. Seu funcionamento é garantido por três elementos estruturais: Casos de Teste, Asserções (que validam se o resultado é o esperado) e Ciclo de Vida (Fixtures) para preparação e limpeza do ambiente.
- **Padronização Triple A:** A escrita interna do código de teste deve seguir o padrão profissional *Arrange* (Preparação), *Act* (Ação) e *Assert* (Verificação) para manter o código limpo e lógico.
- **Documentação e Infraestrutura:** A automação profissional não existe sem um ambiente bem configurado (via Composer, Docker, etc.) e sem a correta documentação e mapeamento dos Casos de Teste na Engenharia de Software, o que garante a rastreabilidade de bugs.